

面向 LLM / Agent Systems 科研的深度学习零基础课

第 03 讲

实验设计：Dot Product、方向、尺度与微型 Attention 打分

实验目标不是“会调 API”，而是用可复现实验验证本讲四个判断：点积的计算规则、方向效应、尺度效应，以及一个 Query 对多个 Key 的打分。

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1 实验问题

本实验回答四个问题：

1. **计算规则**：NumPy 的 `np.dot` 是否等于“逐项相乘再求和”？
2. **方向**：同方向、垂直、反方向时，点积的符号怎样变化？
3. **尺度**：如果方向不变，只把向量放大，点积会怎样变化？Cosine Similarity 又怎样变化？
4. **Attention 雏形**：一个 Query 对多个 Key 做点积时，能否得到一组 score？矩阵写法是否与逐个计算一致？

2 实验假设

运行代码前先写预测：

- H1: $[1, 2, 3] \cdot [4, 5, 6] = 32$;
- H2: 同方向点积为正，垂直为 0，反方向为负；
- H3: 把 $[1, 1]$ 放大为 $[10, 10]$ ，与同一个 x 的点积放大，但 cosine similarity 不变；
- H4: 三个 Key 与一个 Query 会得到 Shape (3,) 的 score 向量；逐个 loop 与矩阵运算结果一致。

科研习惯

先写假设再运行代码，可以避免“看见什么结果就临时编一个解释”。未来做论文实验时，这种习惯会直接帮助你区分 hypothesis、observation 和 interpretation。

3 实验环境

最低要求：

- Python 3.x;
- NumPy;
- PyTorch 可选，仅作为核对，不安装也不影响本讲。

运行：

```
python lesson03_dot_product_experiment.py
```

4 实验 1：手工逻辑与 `np.dot` 是否一致

代码核心：

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])

products = a * b
manual = products.sum()
auto = np.dot(a, b)
```

预期：

$$products = [4, 10, 18]$$

$$manual = 32, \quad auto = 32$$

观察重点：逐元素乘法输出向量，点积输出标量。

5 实验 2：方向与点积符号

固定：

$$q = [1, 0]$$

测试：

$$[2, 0], [0, 3], [-2, 0]$$

预期点积：

$$2, 0, -2$$

同时记录 cosine similarity，应分别接近：

$$1, 0, -1$$

6 实验 3：尺度效应

固定：

$$x = [1, 1]$$

比较：

$$y_1 = [1, 1], \quad y_2 = [10, 10]$$

预期：

$$x \cdot y_1 = 2$$

$$x \cdot y_2 = 20$$

但二者 cosine similarity 都约为 1。

要验证的核心机制

点积的变化既可能来自方向变化，也可能来自长度变化。因此“点积大 = 更相似”必须结合上下文理解。

7 实验 4：一个 Query 对多个 Key

设置：

$$q = [1, 2]$$
$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

逐个算：

$$[1, 2, 3]$$

矩阵形式：

$$Kq = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

代码中同时使用：

```
scores_loop = np.array([np.dot(query, k) for k in keys])
scores_matrix = keys @ query
```

要求两者完全一致。

与 LLM / Transformer / Agent Systems 的连接

这就是 Attention 的最小雏形：一个 Query 与多个 Key 产生多个 score。真正 Transformer 中会把所有 Query、所有 Key 都堆成矩阵，一次计算 QK^T 。第 04 讲会正式解释为什么矩阵乘法就是“大批量点积”。

8 实验 5：可选 PyTorch 对照

如果环境安装了 PyTorch，代码会自动验证：

```
torch.dot(ta, tb)
tk @ tq
```

本实验不要求理解 autograd、nn.Module 或 GPU。这里只用 PyTorch 证明：框架换了，底层数学关系没有变。

9 记录表

实验	预测	实际	是否支持假设/解释
手工点积 vs np.dot	32 = 32		
同/垂直/反方向	+ / 0 / -		
尺度放大	dot 增大，cos 不变		
多个 Key 打分	[1,2,3]		

10 实际结果的判读方法

若结果符合预测，不代表“理论绝对正确”，而代表**这些具体测试案例与理论预期一致**。未来科研中要避免用很少的例子过度外推。

若结果不一致，按顺序检查：

- 1. 输入数字是否写错；
- 2. Shape 是否正确；
- 3. 是否把逐元素乘法误当点积；
- 4. 是否出现零向量导致 cosine 分母为 0；
- 5. 数据类型是否造成意外行为。

11 实验后必须写出的四句结论

请自己完成：

- 1. Dot Product 的计算步骤是：_____
- 2. 同方向/垂直/反方向时，点积通常分别表现为：_____
- 3. Dot Product 与 Cosine Similarity 的关键区别是：_____
- 4. 一个 Query 对多个 Key 打分可以被组织为：_____

本节检查点

实验完成标准：你不仅能运行代码，还能在不看代码时解释每一个输出为什么出现，并能提前预测 Shape。